

Pyramid: A General Framework for Distributed Similarity Search

Shiyuan Deng, Xiao Yan, Kelvin K.W. Ng, Chenyu Jiang, James Cheng

Department of Computer Science

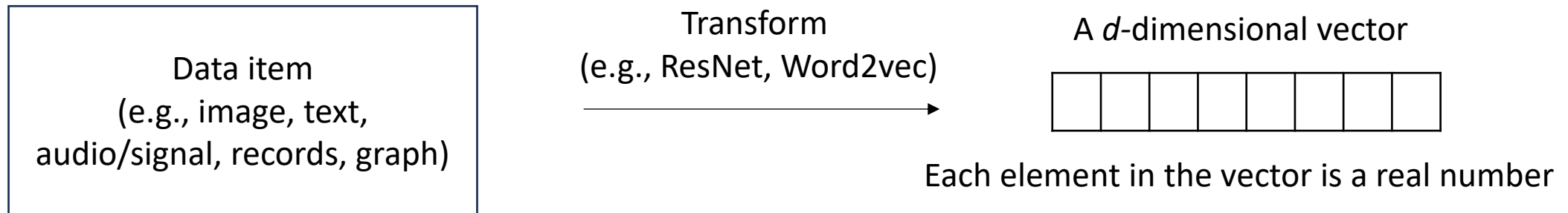
The Chinese University of Hong Kong

Shatin, Hong Kong

{sydeng7, xyan, kwng6, cyjiang7, jcheng}@cse.cuhk.edu.hk

Background

- Vector embedding is a numerical representation of a data item in a d -dimensional space
- The problem: top k search in a vector dataset (also known as nearest neighbor search, k -NN)
- Exact solutions are computationally expensive
- Approximate solutions (ANN) often suffice in practice

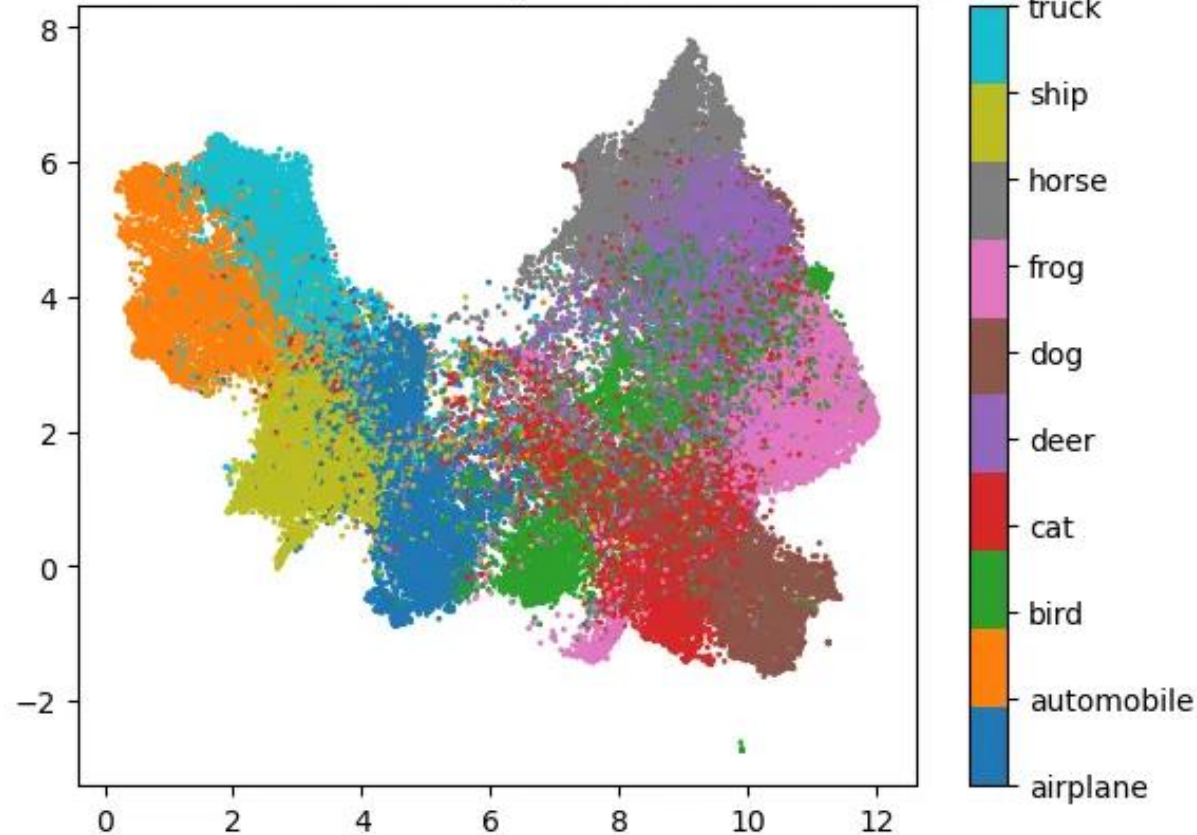


- A vector database/dataset is a collection of vectors $\mathcal{X} = \{x_i \in \mathbb{R}^d | i = 1, \dots, n\}$
- Top k similarity search finds the k items that is most “similar” to a query q in X
- Popular similarity functions $s(q, x)$
 - Euclidean distance
 - Angular distance (e.g., cosine similarity)
 - Inner product

Background

- Vector embedding is a numerical representation of a data item in a d -dimensional space
- UMAP, T-SNE etc. dimensionality reduction techniques are useful for tasks such as 2D/3D visualization

UMAP Visualization of CIFAR-10 with pre-trained Resnet50 embeddings



Find top k similar images in the CIFAR-10 dataset

Background

- Popular similarity/distance functions (assuming an n -dimensional feature space)
 - Euclidean distance

$$\|\mathbf{A} - \mathbf{B}\| = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

- Angular distance

Cosine similarity,

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Cosine distance, $1 - \cos(\theta)$

- Inner product

Dot product, $\mathbf{A} \cdot \mathbf{B} = \sum_{i=1}^n A_i B_i = \|\mathbf{A}\| \|\mathbf{B}\| \cos(\theta)$

- A larger similarity value indicates that two items are more similar.

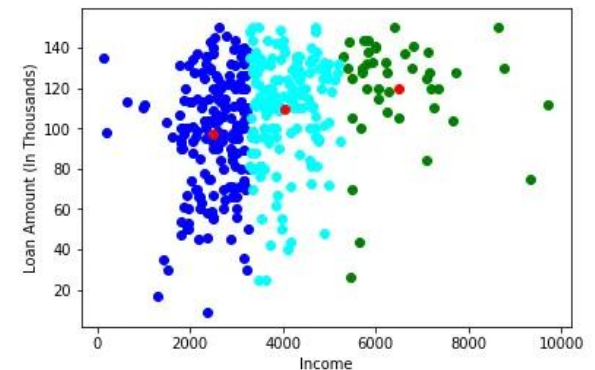
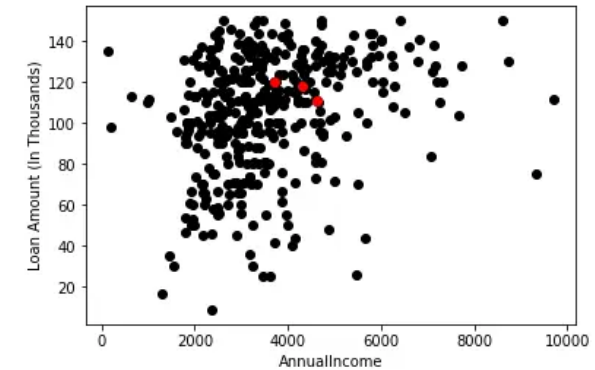
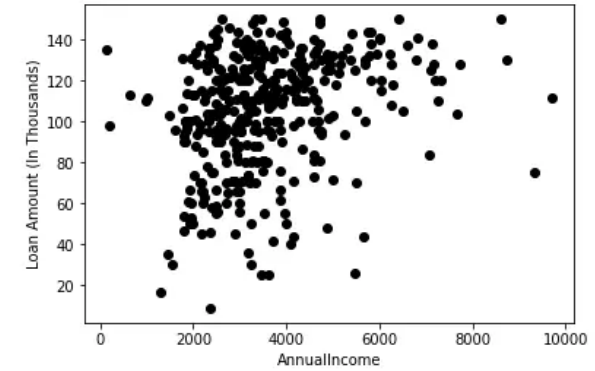
Well-known vector databases / analytics tools

- FAISS
- Qdrant
- Pinecore
- Milvus (Zilliz)
- Pgvector (Postgre)
- RediSearch (Redis)
- OpenSearch (Elasticsearch)

Background

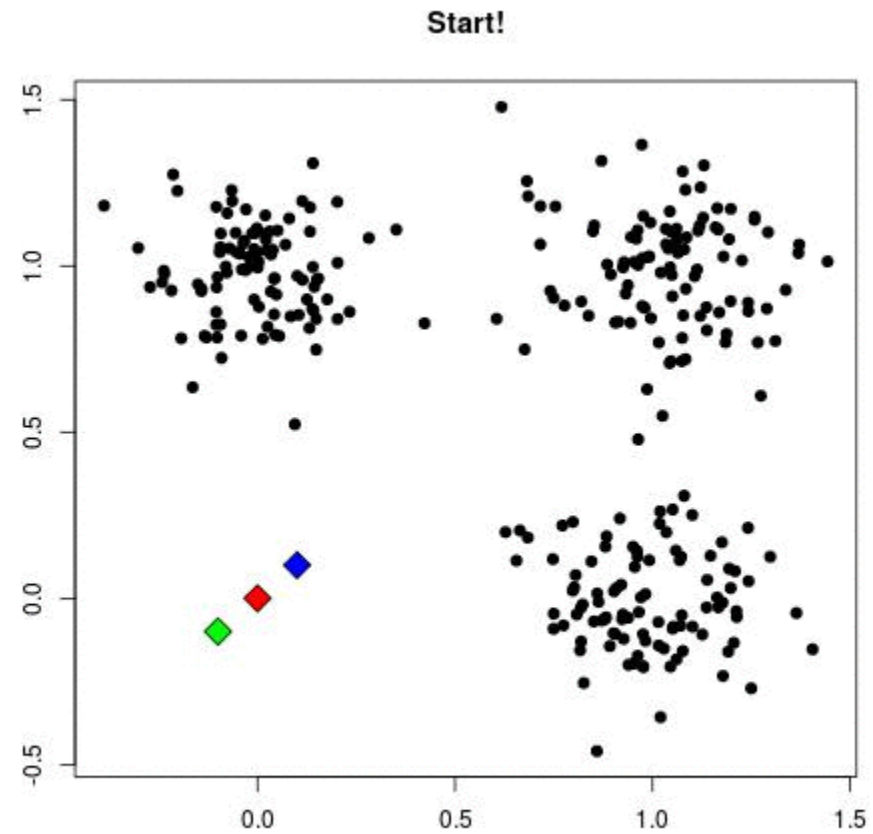
K-means clustering

- Randomly select k centroids (initial cluster centers)
- Repeat until convergence
 - For every point in the input dataset, compute distance from each centroid
 - For every point in the input dataset, assign a point to the cluster of the centroid it is closest to
 - For each cluster, calculate the mean of all data points and select it as the new cluster centroid*
 - For all clusters, check whether at least one centroid has changed
- A centroid can be a non-data point in the feature space
- *In each iteration, move each centroid to minimize the distance to data points in its cluster
- Conditions for convergence - e.g., no centroid change was observed in last r iterations



Background

K-means clustering



Overview

- Existing similarity search algorithms can be roughly grouped into four categories, i.e.,
 - Tree-based methods [12]-[14],
 - Locality sensitive hashing (LSH) based methods [2], [15], [16],
 - Vector quantization-based methods [17]-[19] and
 - Proximity graph-based methods [20]-[22].
- Well known tools such as FAISS [29] uses vector quantization techniques to reduce memory consumption. However, the quantization error introduced by the compression process often negatively impacts the quality of the search results.
- Among existing similarity search approaches, the proximity graph-based methods were shown to provide superior recall-time performance in a number of empirical studies.
- Recall-time performance measures the recall achieved within a given query processing time and higher recall indicates better performance.

Overview

- Apart from producing high quality results, a similarity search framework needs to fulfill three additional requirements for production use, i.e.,
 - High query processing throughput,
 - Low query processing latency, and
 - Robustness.
- Query processing throughput is the number of queries the framework can handle in unit time.
- Query processing latency measures time taken to process a query and online applications typically require a query processing latency in the order of several milliseconds.
- One of the main challenges of similarity search on large-scale datasets is the memory required to store the raw data and index data structures, which could easily exceed the capacity of a single machine.
- For example, the SIFT1B dataset [28], which contains 1 billion 128-dimensional SIFT descriptors (vectors) of images, takes up 512GB of memory for holding the raw data if each feature is stored as a floating-point number.

Overview

Proximity graph-based methods

- In a proximity graph, each item is connected to a small set of items that are most similar to it in the dataset.
- A graph-based methods usually conduct similarity search by a walk on the proximity graph.
- With the proximity graph, graph-based methods can model the fine-grained neighboring relation among items and avoid checking dissimilar items for query processing, which explains their good performance.
- The proximity graph-based techniques (e.g., HNSW) take up a large amount of memory (often comparable with the size of the raw data) as they need to maintain a neighbor list for every item.

Overview

Contribution summary

- Pyramid is a distributed solution based on HNSW and supports popular similarity functions including Euclidean distance, angular distance and inner product.
- Pyramid solves the deficiency of the naive solution with novel dataset partitioning and query assignment strategies.
- The key idea is to build a much smaller meta-HNSW that captures the structure of the entire dataset, which allows us to build two levels of indices and pinpoint the neighbors of a query efficiently.
- By partitioning the bottom layer of the meta-HNSW, Pyramid assigns dataset items to sub-datasets of roughly equal size and ensures that items in the same sub-dataset are similar to each other.
- By searching the meta-HNSW, Pyramid quickly identifies the sub-datasets that are likely to contain the neighbors of a query and involves only these sub-datasets in query processing without hurting the quality of the search results.
- We tested Pyramid on three large-scale datasets, Deep500M, SIFT500M and Tiny10M. The results show that Pyramid provides high quality search results and the precision can easily reach 90% for top 10 Euclidean nearest neighbor search.
- The throughput of Pyramid is over 2x compared with a naive solution that randomly partitions a dataset among the machines and builds an HNSW for each machine.
- Comparing with the famous FLANN library [25] that uses a tree-based method for distributed similarity search, Pyramid provides a throughput that is over 100x higher and achieves better precision for the search results. Pyramid is able to keep the query processing latency within 2-3ms.

Background

HNSW (Hierarchical Navigable Small World)

- The HNSW proximity graph has multiple layers.
- The bottom layer (layer 0) contains all items in the dataset.
- The items in each upper layer are sampled uniformly from its previous layer.
- The number of items reduces as you go up the layers.

Index construction in HNSW

- Each layer of HNSW is an approximation of the k -nearest neighbor graph (KNN graph).
- An item is connected to its approximate top M neighbors in that layer, where M is a user-specified number that controls the size of the graph.

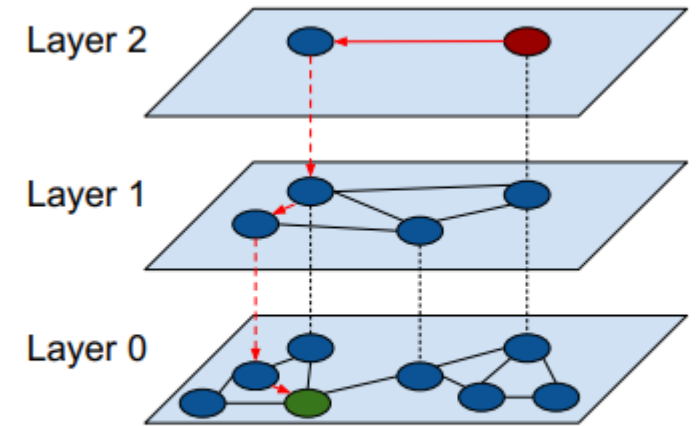


Fig. 1. An illustration of an HNSW Graph

Background

Query processing in HNSW

- C is the candidate queue and W keeps the best results encountered so far.
- The graph walk starts at a fixed entry vertex at the top layer (the red vertex in the figure).
- Before reaching the bottom layer, graph walk is conducted with a search factor of $l = 1$, which is also called greedy graph walk (without backtracking). Note that l is a used defined parameter which also that controls the size of W .
- Greedy graph walk moves to the best neighbor (most similar to the query) of the current vertex at every step and stops when the current vertex is more similar to the query than all its neighbors.
- The stopping vertex in an upper layer is used as the starting vertex for graph walk in the layer below it.
- For the bottom layer, graph walk is usually conducted with a search factor $l > 1$, which is similar to beam search with backtracking and makes the walk less likely to be trapped at local optimal. (Beam search keep track of at most p candidates at each level. Candidates are not guaranteed to be the p best at each level.)
- The upper layers of HNSW contain uniformly sampled items from the dataset and they allow graph walk to take large steps and quickly approach the neighborhood of the query.
- Using a large search factor l , graph walk on the bottom layer allows good exploration of the neighborhood of the query to identify the true neighbors.

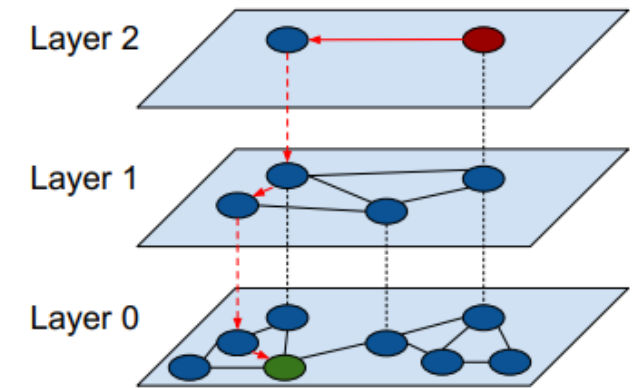


Fig. 1. An illustration of an HNSW Graph

Background

HNSW (Hierarchical Navigable Small World)

- HNSW is an improvement of the ordinary KNN graph, which contains a single layer and each item is connected to its top neighbors.
- In a KNN graph, graph walk can only take small steps as the connections are local.
- If the starting vertex is far from the neighborhood of the query (similar points), graph walk needs a large number of steps to reach the true neighbors, which negatively impacts performance.
- The upper layers of HNSW contain uniformly sampled items from the dataset and they allow graph walk to take large steps and quickly approach the neighborhood of the query.
- Using a large search factor ($l > 1$), graph walk on the bottom layer allows good exploration of the neighborhood of the query to identify the true neighbors.

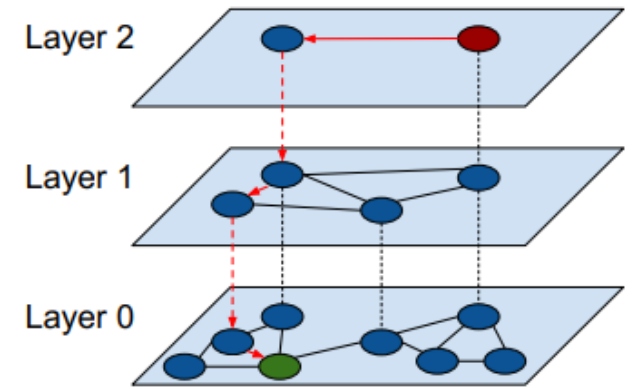


Fig. 1. An illustration of an HNSW Graph

Background

HNSW (Hierarchical Navigable Small World)

- The items in the dataset are inserted sequentially into the graph to build the HNSW index.
- For an item x , the highest layer (u) it can appear in is first generated by an exponential distribution.
- During index construction, then graph search is conducted using x as the query and the only difference from Algorithm 1 (query processing procedure) is that large search factor (l) is used for all layers below $u + 1$.
- In each of these layers, graph search is used to find the top M neighbors of x and x is connected to them using directed edges.
- It is reported that the search complexity of HNSW scales with $O(\log n)$ with n being the cardinality of the dataset [22], which is favorable for large datasets.
- Index construction of HNSW is also efficient as it does not require to build an exact KNN graph at each layer $O(n \log n)$.

Proposed solution

Naïve distributed HNSW

- A straightforward solution is to randomly partition a dataset among workers in a cluster and build an independent HNSW graph on each worker.
- A query q is distributed to all workers and each worker processes the query with its own HNSW graph using Algorithm 1 (query search procedure).
- The final search results are obtained by merging and re-ranking the partial results reported by the workers.
- However, invoking computation on all workers would result in low query processing throughput (e.g., large number of partial results).
- If a query is handled by only some rather than all of the workers, query processing throughput can be improved as each query invokes less workload.
- This is the main motivation of Pyramid's design, which is achieved by dataset partitioning and query assignment.

Proposed solution

Pyramid - index construction

- In the index building phase, Pyramid partitions the dataset into sub-datasets containing items similar to each other and assigns each sub-dataset to a worker.
- Due to the partitioning, some sub-datasets are likely to contain the neighbors for a query while others are not.
- It is sufficient to handle the query by the workers holding these potential sub-datasets and the other workers do not need to be involved.
- Both dataset partitioning and query assignment are conducted with a small meta-HNSW built on samples from the dataset.

Proposed solution

Pyramid - index construction

- As the original dataset X may be very large, we first sample a small dataset X' with size n' from it.
- Then, k-means clustering with m centers is conducted on the sample dataset X' and we assign a weight to each k-means center, which is set as the number of items it has from X' .
- The meta-HNSW graph G_m is built on these m k-means centers using Algorithm 2 (graph construction).
- We partition the bottom layer (layer 0, which is a proximity graph) of G_m into w balanced graph partitions (in the sense that each graph partition has similar total vertex weights) and try to minimize the number of edges across the graph partitions.

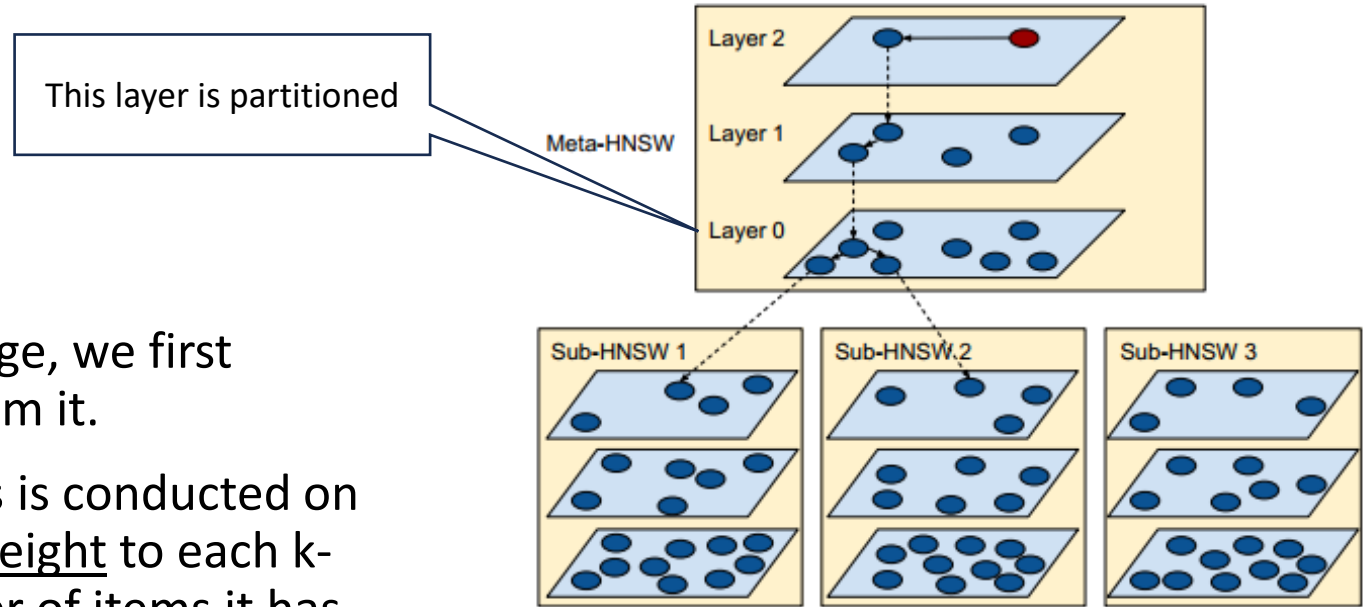


Fig. 2. An Illustration of Pyramid

Meta-HNSW is replicated on all workers.

Proposed solution

Pyramid - index construction

- Finally, for each item x in the original dataset X , we assign it to sub-dataset X^i according to its most similar item in G_m .
- For each sub-dataset X^i , we build a sub-HNSW G_i independently.
- There is a one-to-one mapping between the sub-datasets and the graph partitions (bottom layer of G_m) and the i -th partition corresponds to sub-dataset X^i .

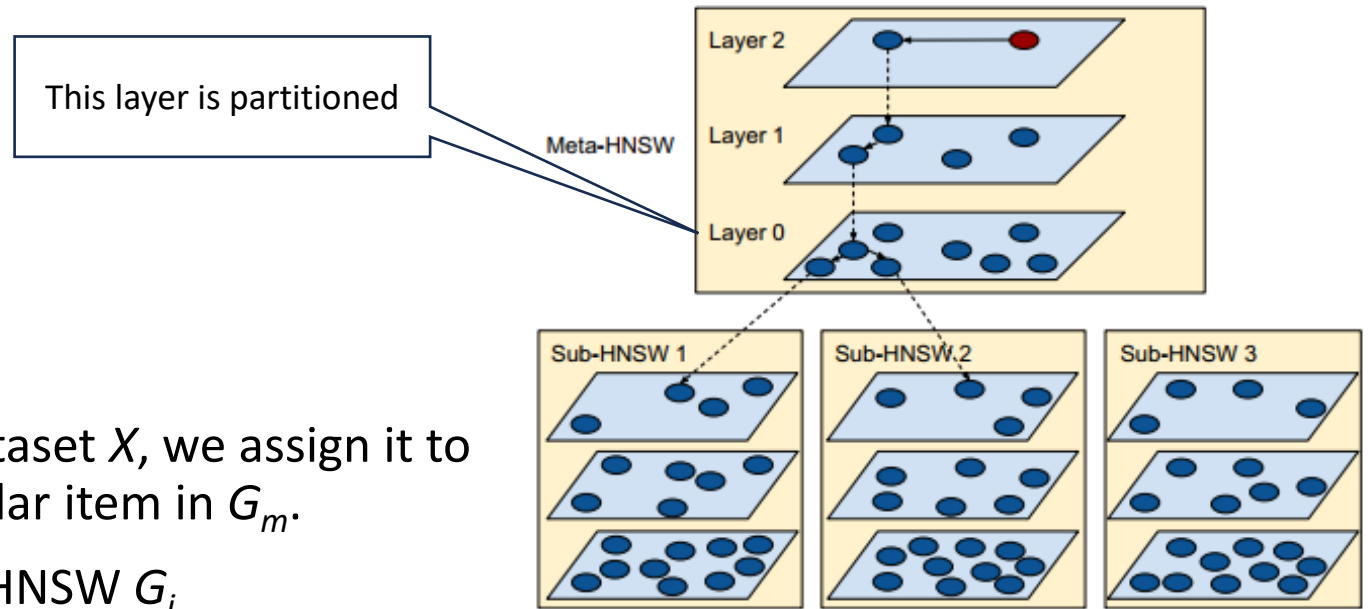


Fig. 2. An Illustration of Pyramid

Meta-HNSW is replicated on all workers.

Proposed solution

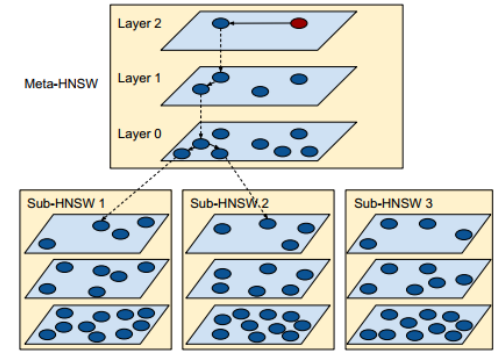


Fig. 2. An Illustration of Pyramid

Pyramid - index construction

- The design of Algorithm 3 (distributed index construction) is a joint consideration of efficiency, load balancing and statistical stability.
- As the original dataset is large, we use a smaller sample dataset X' as its surrogate to speed up the index building process.
- We set the weight of a vertex in G_m as the number of items it has from X' and ensure that the partitions of G_m have similar total vertex weights.
- There may be scenarios that some items in X are hot (more likely to be accessed by queries).
- In this case, we can set the weight of a vertex in G_m as the frequency it appears in the top k similarity search results of the queries for load balancing.
- We do not directly sample m items from the dataset to build the meta-HNSW as m may be small and a small sample may not reflect the distribution of the entire dataset.
- We observed that a small meta-HNSW is already sufficient for good performance and a large meta-HNSW is not favorable as it prolongs the query processing time.
- Therefore, we conduct k-means on a larger sample (i.e., X' with n' items) of the dataset to obtain the vertices in the meta-HNSW for statistical stability.

Proposed solution

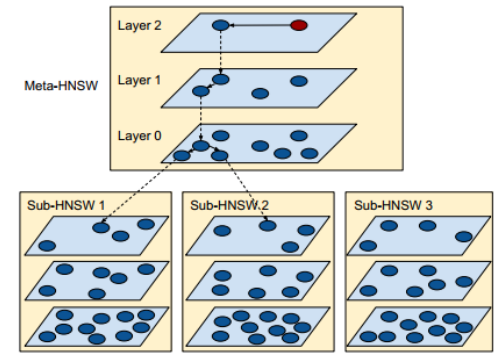


Fig. 2. An Illustration of Pyramid

Pyramid - index construction

- Workers conduct distributed k-means with m centers together.
- The centers are kept in one of the workers for meta-HNSW construction and graph partitioning.
- When graph partitioning finishes, the worker also builds a one-to-one mapping between the graph partitions (and the sub-dataset indices) and the workers.
- After that, the meta-HNSW is broadcast to all workers along with related data structures.
- The workers decide for each item in its local dataset which sub-dataset it belongs to using the meta-HNSW and shuffles (data transfer) the items to their destination workers in parallel.
- When data shuffle finishes, each worker builds an HNSW on its own sub-dataset.

Proposed solution

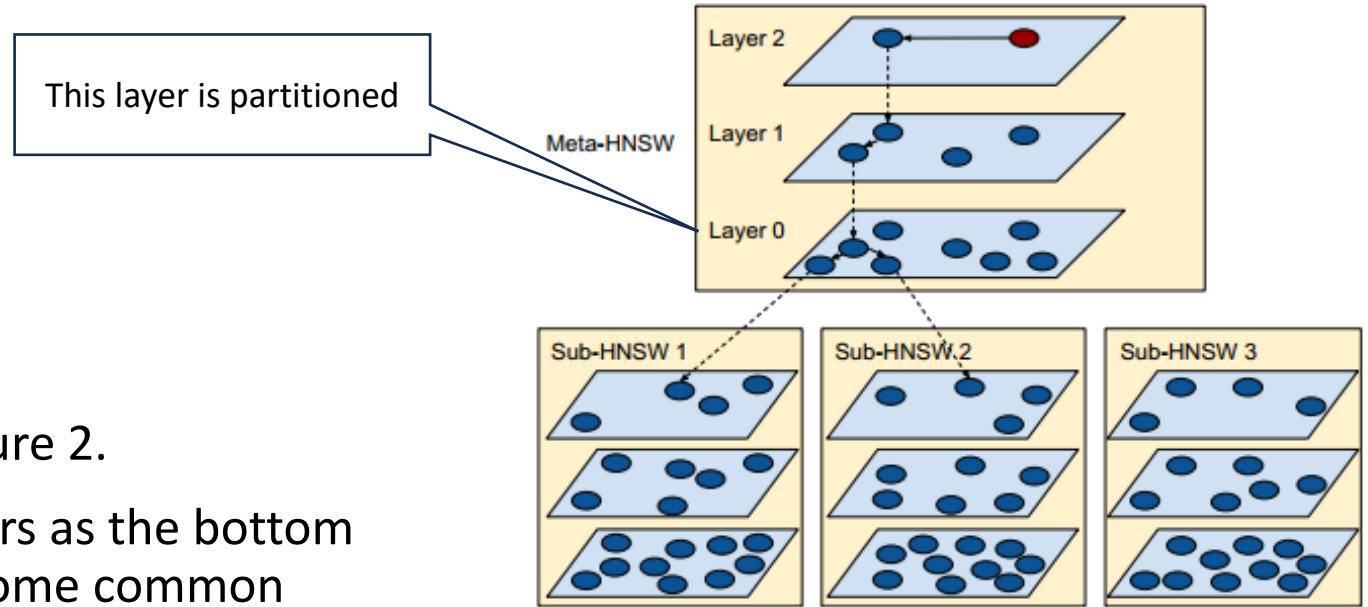


Fig. 2. An Illustration of Pyramid

Pyramid - index construction

- We give an illustration of Pyramid in Figure 2.
- If we treat the sub-HNSWs on the workers as the bottom layer, the meta-HNSW is equivalent to some common upper layers of the sub-HNSWs.
- By searching the meta-HNSW, we can quickly identify the sub-HNSWs that are likely to contain the neighbors of a query.
- This is analogous to the upper layers of an HNSW, which help quickly approach the neighborhood of the query.
- By using the bottom layer of the meta-HNSW for dataset partitioning, we ensure that items in the same sub-dataset are similar to each other.

Proposed solution

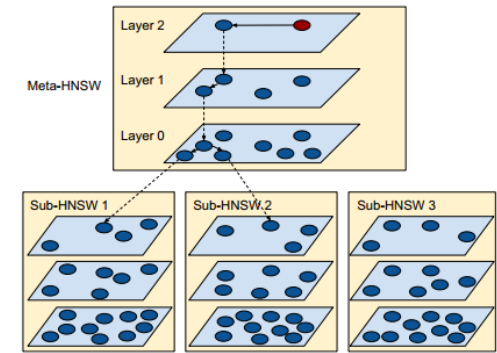


Fig. 2. An Illustration of Pyramid

Pyramid - query processing

- Query processing starts by finding the top K neighbors of the query in the meta-HNSW graph G_m , which can be done very efficiently as the meta-HNSW is usually very small.
- For each graph partition of the bottom layer of G_m , if it contains one or more of these K neighbors, the query will be dispatched to its corresponding sub-dataset (sub-HNSW).
- Here, K is called the branching factor as it controls how many sub-datasets a query will be forwarded to and a larger K means more sub-datasets will be involved.
- For each sub-dataset that receives the query q , HNSW-based graph walk is conducted to find the top k neighbors of q (within a sub-HNSW).
- The final search results are obtained by selecting the top k neighbors from the partial results returned by the sub-datasets (multiple sub-HNSWs).
- The query processing algorithm ensures that only some of the sub-datasets are involved in answering a query, which helps to achieve high query processing throughput.

Proposed solution

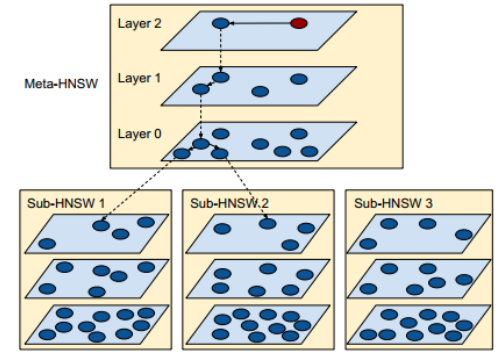


Fig. 2. An Illustration of Pyramid

Pyramid - query processing

- When a query q is received, q is assigned to a random worker as the coordinator.
- The coordinator searches the meta-HNSW with the query and decides the sub-datasets that will be involved in processing the query according to the search results.
- Then, the query is dispatched to the corresponding workers and the workers search their own HNSW with the query.
- Each involved worker returns k tuples of $(item\ id, similarity\ score)$ to the coordinator.
- When all responses for a query are gathered, the coordinator selects items with the top k similarity scores as the final results.

- Skip section IV.

Evaluation

Performance and quality metrics

- Throughput - the number of queries that can be processed by per second.
- Latency - the interval between the time a system receives a query and the time the system returns the top k similarity search results.
- Precision - for top k similarity search, an algorithm is allowed to return k items. If k' of these k items belong to the ground-truth top k neighbors of a query, the precision is said to be k'/k . Precision measures the quality of similarity search results and higher precision means better quality.

Evaluation

Methodology

- We searched for the top 10 neighbors of a query in the experiments.
- For latency, we report the 90-th percentile instead of the average, which models the worst-case performance of the system and is more critical to online applications.
- For the meta-HNSW and the sub-HNSWs in Pyramid, we set their parameters according to the recommendation in the HNSW paper [22] and did not fine-tune them.
- Specially, the maximum out-degree of an item is set to 32 for the bottom layer graph. The maximum out-degree is set to 16 for the other layers.
- The search factor f for graph walk on the bottom layer was set to 100.
- As the cluster contains 10 machines, we use 10 sub-HNSWs in the experiments.

Evaluation

Methodology

- Recall that the meta-HNSW has two main parameters, i.e.,
 - Size m (the number of vectors in the bottom layer graph) and
 - Branching factor K (the number of top neighbors in the meta-HNSW that are used to choose the sub-HNSWs for a query).
- We experimented with a meta-HNSW of size 1,000, 10,000 and 100,000 and
- Branching factor value of 1, 5, 10, 20, 50 and 100.

Evaluation

TABLE I
DATASETS

Name	# item	# dimension	size (GB)
Deep500M	500,000,000	96	192
SIFT500M	500,000,000	128	256
Tiny10M	10,000,000	384	15.4

Datasets

- Deep500M was sampled from the Deep1B dataset and contains descriptors of images.
- The SIFT500M dataset was sampled from the SIFT1B dataset and contains 128-dimension SIFT descriptors of images.
- The Tiny10M dataset was sampled from the Tiny80M dataset and contains the GIST descriptors of the Tiny images.
- Both Deep500M and SIFT500M contain items with very similar Euclidean norm, conducting Maximum Inner Product Search (MIPS) on them is not interesting as it is equivalent to Euclidean nearest neighbor search.
- In contrast, Tiny10M has a wide spread in Euclidean norm distribution and is more suitable for testing the performance of MIPS.

Evaluation

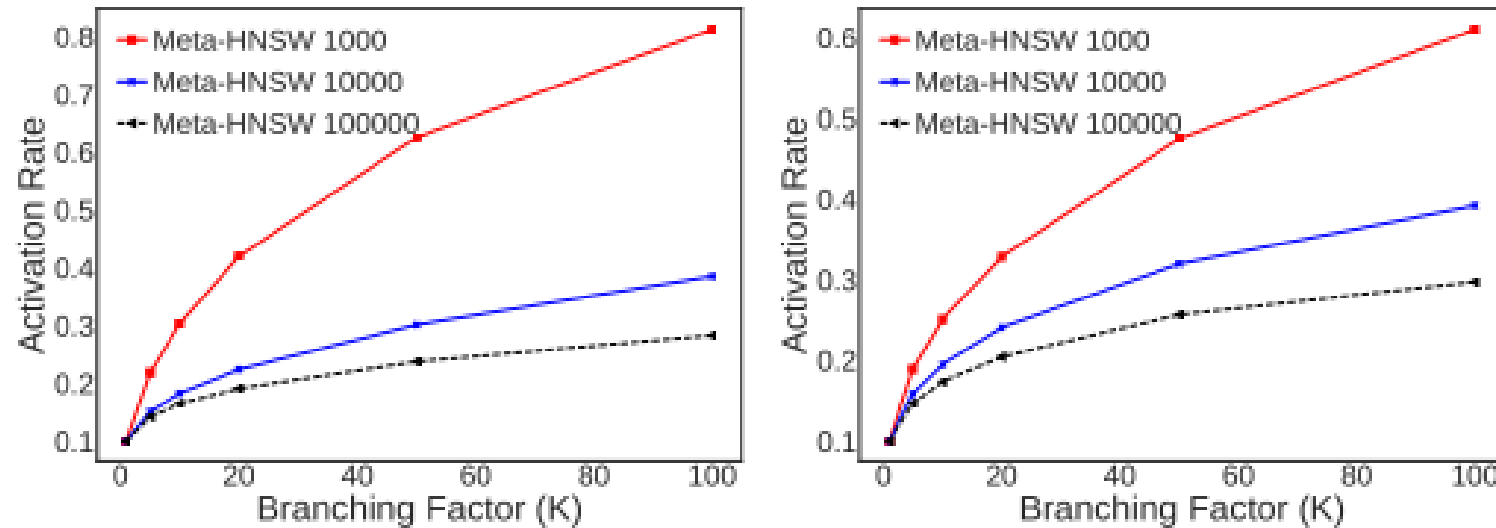


Fig. 5. Access rate vs. branching factor for Deep (left) and SIFT (right)

- Average access rate of the queries under different branching factor, where the access rate is the fraction of sub-HNSWs that are accessed for processing a query.
- Under the same meta-HNSW size, the access rate increases with K as a larger K means that more neighbors in the meta-HNSW are used to choose the sub-HNSWs .
- For the same K , a larger meta-HNSW size results in a lower access rate.

Evaluation

- The precision is higher for a smaller meta-HNSW size under the same branching factor.
- This is because more sub-HNSWs are accessed to process a query for a smaller meta-HNSW according to Figure 5.

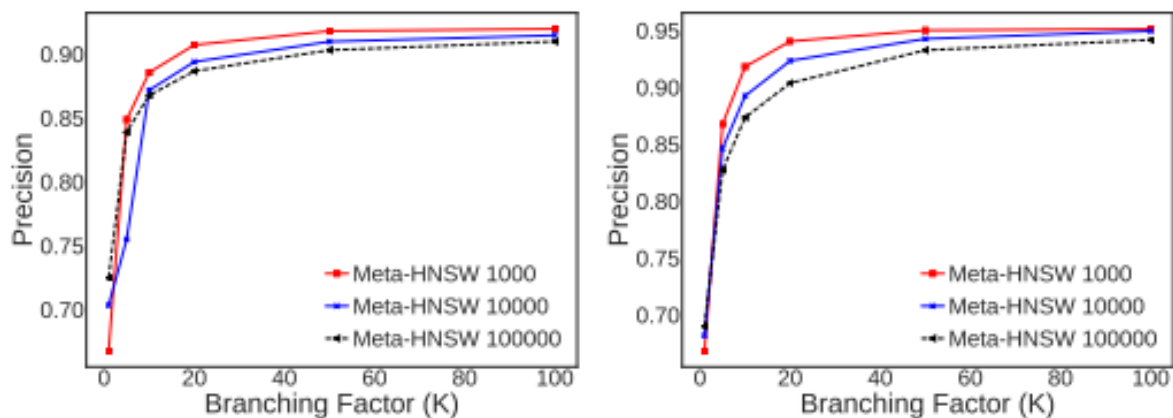


Fig. 6. Precision vs. branching factor for Deep (left) and SIFT (right)

- Throughput consistently drops when K increases.
- This is because a larger K results in a higher access rate, which means that more sub-HNSWs are searched to answer a query.
- Although the access rate is lower for a larger meta-HNSW size under the same K according to Figure 5, meta-HNSW 100,000 does not always achieve higher throughput.
- This is because searching a larger meta-HNSW has higher latency.
- A large meta-HNSW is not required to achieve the optimal performance, which is favorable as the meta-HNSW needs to be replicated on every coordinator and a large meta-HNSW will take up a larger amount of memory.

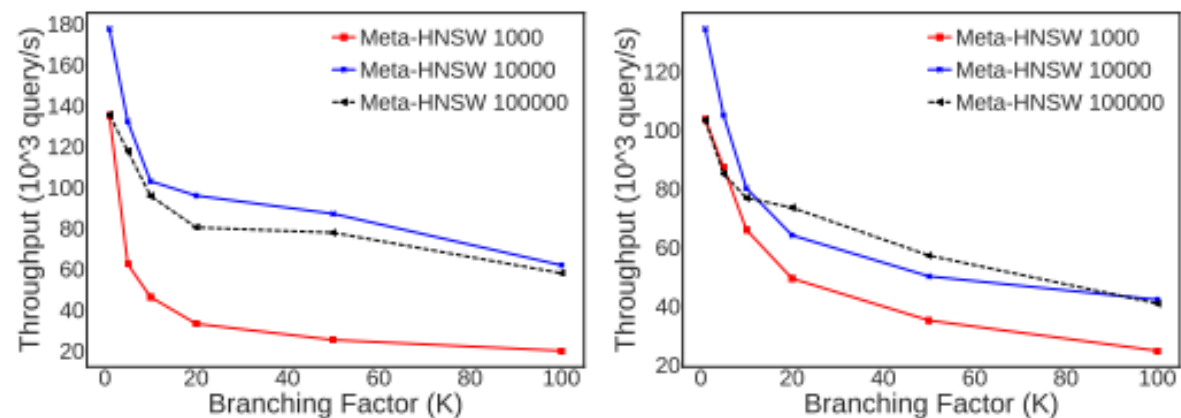


Fig. 7. Throughput vs. branching factor for Deep (left) and SIFT (right)

Evaluation

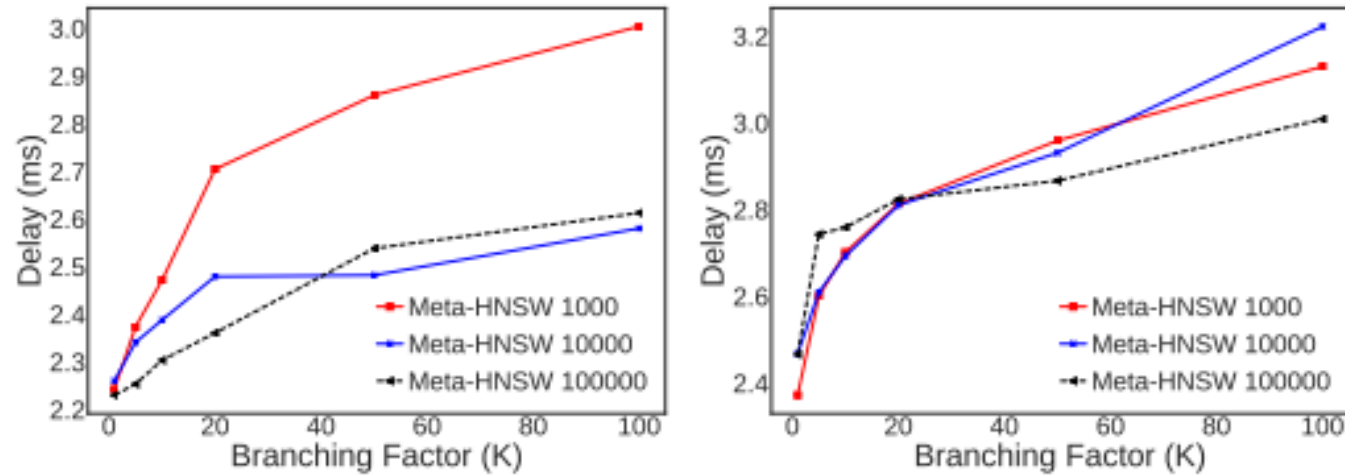


Fig. 8. 90th PCT latency vs. branching factor for Deep (left) and SIFT (right)

- The results show that the latency increases with the branching factor K .
- This is because a coordinator needs to aggregate the results from more partitions due to the large access rate under a large K and the latency depends on the maximum latency in these partitions.
- A large meta-HNSW requires longer time for searching the meta-HNSW.

Evaluation

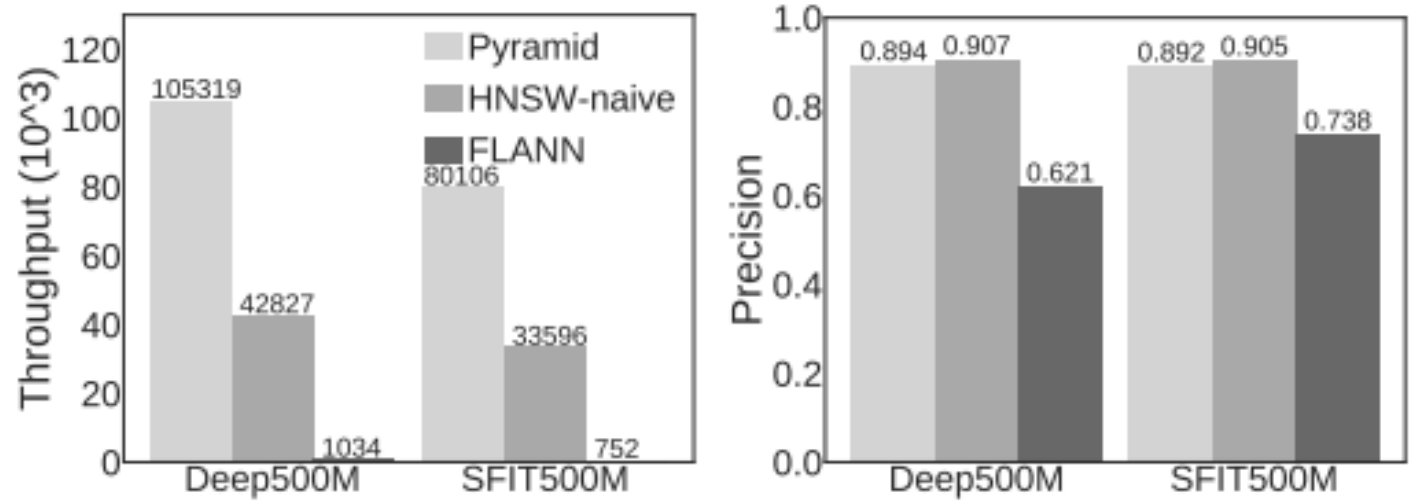


Fig. 9. Throughput and precision comparison of the systems

Comparison with other methods

- To enable a fair throughput comparison between Pyramid and HNSW-naive, we adjusted their query processing parameters (branching factor K and search factor l for Pyramid and search factor l for HNSW-naive) to achieve a precision approximately 90%.
- As it is difficult for FLANN to achieve a 90% precision, we report both its precision and throughput under the setting recommended in [25].
- As each query generates less workload in Pyramid, its throughput is much higher than HNSW-naive.
- Moreover, Pyramid and HNSW-naive outperform FLANN in both throughput and precision due to the algorithmic advantage of HNSW over the tree-based method adopted in FLANN.

Evaluation

Cost of index construction

- It is worth noting that the better performance of Pyramid comes at the cost of more expensive index building.
- For the Deep500M dataset, Pyramid took about 162 minutes for index building using 10 machines,
 - with 31 minutes for metaHNSW construction, 87 minutes for dataset partitioning and 44 minutes for sub-HNSW construction.
- As a comparison, HNSW-naive took 53 minutes to build the index,
 - with 14 minutes for dataset partitioning and 39 minutes for sub-HNSW construction. The longer index building time of Pyramid is mainly caused by dataset partitioning using the meta-HNSW, which needs to search the meta-HNSW with every item.
- For FLANN, index building is extremely fast and took only 38 seconds.
- This suggests Pyramid is suitable for applications where dataset updates are not frequent and online search performance is critical.

Evaluation

- We report the performance of Pyramid on MIPS for the Tiny10M dataset in Figure 10.
- As there are no distributed similarity search solutions that support MIPS, we report the performance of HNSW-naive as the baseline.
- HNSW-naive achieves a precision of 99.7% and a throughput of 12,732 queries per second. In comparison, the throughput of Pyramid is much higher under similar precision thanks to its low access rate. Recall that we allow an item to appear in multiple sub-datasets for MIPS in Algorithm.
- Pyramid achieves a precision of 96.98% under a branching factor of 1, which means that the sub-HNSW in only one of the 10 machines is accessed for processing each query.
- Note that we did not replicate a lot of items across the machines, which would take up a lot of memory.
- For this experiment, we set the replication factor r as 300 and all the sub-HNSWs end up storing a total of 10,060,599 items, which is only 0.6% larger than the original Tiny10M dataset.
- We believe allowing some items to appear in multiple sub-datasets can be an effective measure to reduce the access rate for distributed similarity search and adopting this idea of Euclidean nearest neighbor search will be an interesting direction for future study.

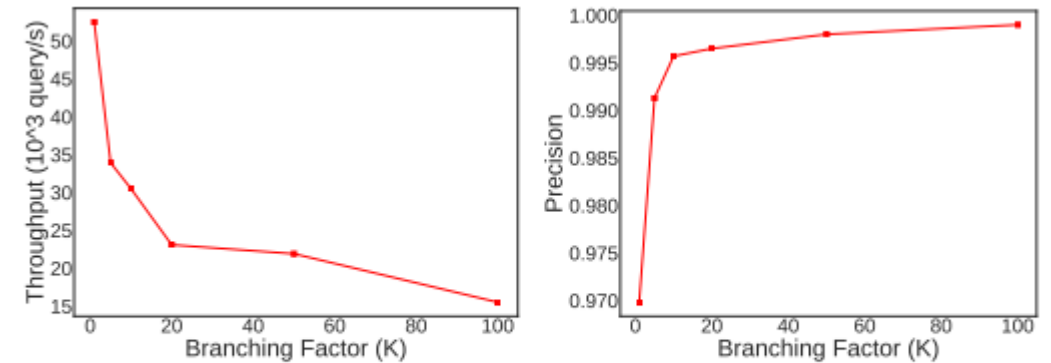


Fig. 10. Performance of Pyramid on MIPS

Evaluation

- For a fair comparison of throughput, we adjusted the query processing parameters (e.g., branching factor K and search factor l) to ensure that both configurations achieved the same precision (80% and 90%).
- We report the results in Figure 11. The throughput with 10 machines is 1.78 and 1.59 times of the case with 5 machines under a precision of 80% and 90%, respectively.
- The reason that Pyramid does not achieve linear scaling could be due to the characteristic of HNSW. (What characteristics?)
- As introduced in Section II, the search complexity of HNSW scales with $O(\log n)$ with n being the cardinality of the dataset.
- To achieve the same precision, the 5-machine configuration needs to access a smaller number of sub-HNSWs but the number of items in each sub-HNSWs is larger comparing with the 10-machine configuration.
- As the search complexity of HNSW increases slowly with the dataset cardinality, the 5-machine configuration has a smaller per-query complexity than its 10-machine counterpart because the influence of using a smaller number of sub-HNSWs is dominant.

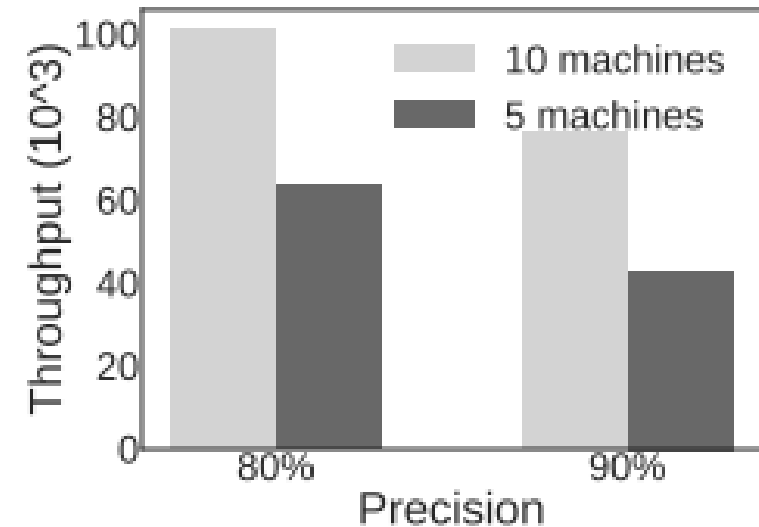


Fig. 11. The scalability of Pyramid